Multi-Agent Hierarchical Reinforcement Learning for Dynamic Energy-Aware Cloud Workflow Scheduling with Deadline Constraints

***Abstract*—As cloud workflow applications expand in scale and complexity, data center energy consumption has grown rapidly. However, energy-saving strategies, including workload consolidation and deferred resource allocation, may increase execution delays and compromise service-level guarantees. These conflicting effects make dynamic workflow scheduling particularly challenging in deadline-sensitive cloud environments. To tackle this issue, this paper establishes a load-aware scheduling model that jointly considers energy consumption and deadline constraints. Based on this model, a multi-agent hierarchical deep reinforcement learning (MAHDRL) framework is proposed to solve the resulting optimization problem. MAHDRL addresses the scheduling complexity by decoupling the decision process into a tripartite sequential pipeline, which consists of a task sequencing agent for global prioritization via adaptive multidimensional weights, a host selection agent for server-level placement, and a virtual machine selection agent for fine-grained resource mapping. By integrating the dueling double deep Q-network with a hierarchical asynchronous update strategy, MAHDRL reduces the effective action space and mitigates frequent global re-sequencing. Extensive experiments on scientific workflows across diverse resource scales demonstrate the effectiveness of the proposed approach. Compared with state-of-the-art algorithms, MAHDRL achieves superior energy efficiency under stringent deadline constraints.**

***Index Terms*—Dynamic workflow scheduling, deadline-constrained, hierarchical reinforcement learning, cloud computing, energy consumption.**

I. Introduction

CLOUD computing has emerged as the core computational backbone for data-intensive applications and high-performance workloads. This infrastructure supports large-scale scientific exploration in domains such as computer vision, earthquake science research, protein-ligand interaction prediction, and natural language processing [1], [2]. However, as cloud workflow applications continue to expand in scale and complexity, their demand for computing, storage, and communication resources has increased substantially, placing growing pressure on cloud data centers [3]. This pressure is further amplified by the rapid development of large-scale artificial intelligence, which has accelerated the growth of energy consumption of data centers. For instance, U.S. cloud data centers are projected to exceed 150 billion kWh by 2030, while Chinese supercomputing platforms such as Tianhe-2 consumed approximately 85,000 MWh annually as of 2024 [4]. Such massive energy demands inevitably incur prohibitive operational costs and carbon emissions. Thus, designing efficient algorithms for green cloud resource scheduling has become an urgent challenge [5]. Nevertheless, reducing energy usage is inherently coupled with the risk of performance degradation. In practice, cloud users typically impose deadline-related quality of service (QoS) requirements through service-level agreements [6]. The inability to satisfy these temporal requirements directly compromises service reliability and results in unacceptable execution delays. For this reason, it is imperative to develop scheduling policies that solve the dynamic energy-aware cloud workflow scheduling problem with deadline constraints (DECWSP-DC), which minimizes global energy consumption subject to deadline constraints.

Extensive literature has primarily focused on static workflow scheduling; however, operational cloud environments are often dynamic [7]. In such dynamic environments, cloud workflow scheduling remains an NP-hard problem [2], [3], [8]. Moreover, this online scheduling problem is further challenged by stochastic arrivals, heterogeneous resource demands, and runtime uncertainties. Consequently, conventional heuristics and meta-heuristics often become less effective, as their fixed rules or predefined search procedures provide limited adaptability to evolving system states [9], [10]. This has prompted growing interest in deep reinforcement learning (DRL) for dynamic cloud resource management [11]. Chi et al. [12] proposed a prioritized experience-based dueling double deep Q-network (D3QN) task offloading scheme to address online task offloading in IIoT-enabled multi-access edge computing environments. Meanwhile, Zhu et al. [13] introduced an online scheduler combining graph neural networks (GNNs) and proximal policy optimization (PPO) to handle dynamic workflow scheduling in hybrid edge-cloud IoT networks.

Despite these advancements, existing DRL approaches still suffer from notable architectural limitations. First, most existing models rely on predefined heuristics for task sequencing, limiting scheduling flexibility. Second, direct mapping from tasks to virtual machines (VMs) causes the state-action space to expand rapidly as the numbers of tasks and VMs increase. Third, deadline constraints and task dependencies further complicate value function estimation, resulting in training instability [14].

Therefore, this study proposes a multi-agent hierarchical deep reinforcement learning (MAHDRL) framework to solve the DECWSP-DC. In MAHDRL, the entire scheduling pipeline is decomposed into three sequential stages: dynamic task sequencing, energy-aware host selection, and fine-grained VM selection. Each phase is handled by a specialized agent, which partitions the complex joint state-action space into manageable sub-problems, thereby reducing exploration complexity. To further stabilize training convergence and prevents frequent global re-sequencing, MAHDRL integrates the D3QN with a hierarchical asynchronous learning strategy. The main

contributions of this paper are summarized as follows.

1) A mathematical model of DECWSP-DC is formulated, capturing energy consumption, workflow tardiness, and state transitions to provide a foundation for optimizing heterogeneous cloud environments under strict deadline constraints.
2) The DECWSP-DC is decomposed into three phases, task sequencing, host selection, and VM selection, and further addressed by a MAHDRL framework that integrates D3QN with state-dependent action masks to confine learning to valid spaces and accelerate convergence.
3) Multidimensional discrete actions are designed to enable the agent to adaptively optimize task weights in real time, allowing for a dynamic response to workflow fluctuations that consistently outperforms static single-attribute heuristics.
4) To bridge hierarchical feedback gaps, a hierarchical asynchronous strategy is introduced to decouple agent updates. The task sequencing agent updates periodically with delayed rewards, while host and VM selection agents update instantly with immediate rewards, filtering noise to ensure stable convergence.

The remainder of this paper is organized as follows. Section II reviews related work on energy-aware cloud workflow scheduling and reinforcement learning-based dynamic scheduling. Section III formalizes the DECWSP-DC and its mathematical model. Section IV details the proposed MAHDRL architecture. Section V outlines the experimental setup and analyzes the evaluation results. Finally, Section VI concludes this paper and discusses future research directions.

II. Related work

A. Energy-Aware Cloud Workflow Scheduling

Driven by massive workloads, cloud computing employs virtualization to accommodate multiple VMs on shared physical servers [15], while the rapid expansion of these infrastructures drives unprecedented electricity demands [3]. As active server operations dictate most of this electricity overhead [16], accurate host power modeling is essential for sustainable workflow scheduling. Structuring these models requires dividing total power into a constant static baseline and a variable dynamic component [17]. To reduce this variable energy consumption, researchers widely adopt dynamic voltage and frequency scaling (DVFS) to align processor frequencies with workload fluctuations. Lin et al. [18] combined task migration with frequency scaling to minimize mobile cloud energy under deadline constraints, while Tang et al. [19] exploited processor slack times for workflow energy reduction. In Singh et al. [20] designed meta-heuristics to balance energy and makespan in DVFS-enabled environments. For heterogeneous architectures, Mei et al. [21] and Wang et al. [22] optimized frequencies for processor-accelerator clusters, while recent studies further extended DVFS to edge-cloud and serverless platforms [23].

However, persistent frequency tuning inevitably degrades response times, exacerbates reliability risks, and incurs hidden execution costs [24], [25], thereby restricting its applicability. To overcome these hardware limits, Xie et al. [5] proposed a load-sensitive energy model driven by the interaction among scheduling, host load, and power consumption, which Li et al. [2] improved using a reinforcement learning framework. Despite establishing a solid theoretical baseline, these models primarily target static environments and inherently neglect execution deadlines. Consequently, they aggressively consolidate tasks onto high-performance servers for energy reduction, causing severe queuing delays that threaten real-time applications. Advancing these existing models, this study extends load-sensitive energy modeling into highly dynamic cloud environments by enforcing strict deadline constraints.

B. Reinforcement Learning-Based Dynamic Scheduling

Since scientific workflow scheduling on heterogeneous resources is NP-hard, extensive studies have explored heuristic and metaheuristic strategies. List heuristics, such as heterogeneous earliest finish time (HEFT) and critical path on a processor (CPOP) [26], exploit directed acyclic graph (DAG) structures to minimize makespan, while Arabnejad and Barbosa [27] proposed the predict earliest finish time algorithm to mitigate localized selection errors. However, list-based methods are inherently greedy and provide limited exploration of the solution space. Consequently, metaheuristics, including particle swarm optimization [28], ant colony optimization [4], fuzzy evolutionary algorithms [8], and estimation-of-distribution methods [5], have been widely adopted to optimize cost, energy, reliability, and efficiency. More recently, Li et al. [2] introduced a Q-learning-based hyper-heuristic to coordinate low-level heuristics for energy optimization.

As traditional heuristics and metaheuristics struggle with stochastic dynamic workflows, DRL has gained increasing attention. Chen et al. [29] employed an actor-critic framework to balance latency and energy efficiency, though DAG precedence constraints were not explicitly handled. Zhang et al. [30] formulated a multi-objective Markov decision process (MDP) and used deep Q-network for Pareto-optimal scheduling, yet the coordination between sequencing and mapping remained limited. For complex workflow patterns, Zhu et al. [13] integrated GNNs with PPO for QoS enhancement, while Lin et al. [31] combined self-attention and action masking for cost optimization on spot instances. Chi et al. [12] further demonstrated the stability of D3QN in online mapping. Nevertheless, most DRL-based studies still prioritize resource selection over explicit ready-task sequencing.

To mitigate the combinatorial explosion of state-action spaces in joint sequencing and mapping problems, recent studies have explored multi-agent reinforcement learning and hierarchical DRL. Liu et al. [32] developed a distributed multi-agent DRL approach for task migration. Li et al. [33] combined heterogeneous graph attention with action decomposition, whereas Sun et al. [34] and Ling et al. [35] decomposed coupled decisions into hierarchical or two-stage subproblems. Similarly, Jayanetti et al. [3] adopted a dual-agent architecture for cross-tier resource allocation, and Cai et al. [7] emphasized hierarchical task decomposition and scheduling in cloud-edge systems. Despite improved scalability, these methods mainly focus on mapping or offloading, leaving ready-task sequencing

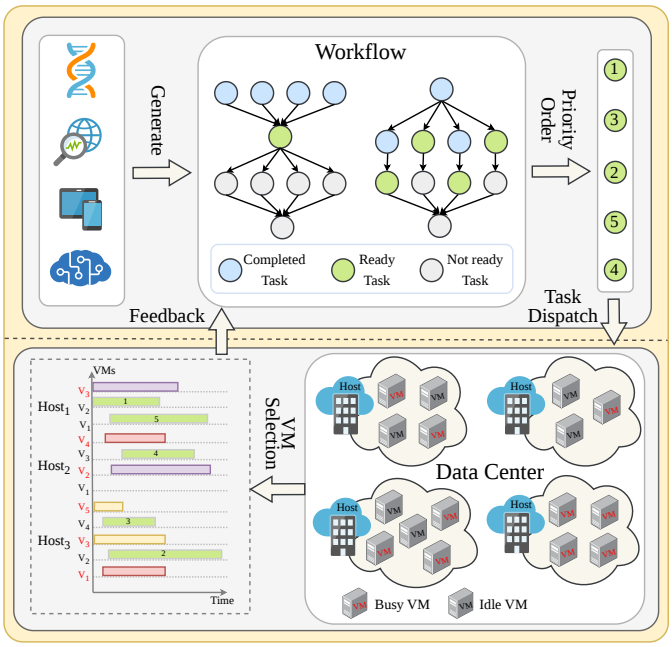


Fig. 1. Proposed scheduling framework.

under DAG dependencies insufficiently addressed. As a result, many DRL-based workflow schedulers still rely on fixed heuristics or loosely decouple sequencing and mapping. He et al. [36] addressed this issue using independent routing and sequencing agents, but its allocate-then-sequence mechanism assigns tasks to device queues in advance. In highly dynamic environments, such premature mapping may introduce state-information lag because resource availability and queue states continue to evolve during task execution.

Given these gaps, especially the reliance on rigid task-ordering rules and the insufficient coordination between task sequencing and resource mapping, a more integrated scheduling paradigm is required. Accordingly, this paper proposes MAHDRL, a hierarchical decision-making framework that coordinates adaptive task prioritization with host and VM selection to improve energy efficiency while maintaining deadline satisfaction in dynamic cloud workflow environments.

III. Problem Formulation

A. System Model and Scheduling Framework

In this study, workflow applications are modeled as DAGs $G = (\mathcal{T}, \mathcal{E})$, where $\mathcal{T} = \{t_1, t_2, \ldots, t_n\}$ denotes a set of n computational tasks, and $\mathcal{E} = \{e_{i',i} \mid t_{i'}, t_i \in \mathcal{T}\}$ represents the directed edges indicating precedence constraints and data dependencies between a parent task $t_{i'}$ and its child task t_i. In dynamic cloud environments, workflows arrive online with uncertain and time-varying patterns, necessitating a scheduler to coordinate interleaved task executions across heterogeneous VMs following a typical scheduling pipeline [12], [13], as illustrated in Fig. 1. These VMs operate under a non-preemptive and single-tasking mode. Upon arrival, tasks enter a shared buffer and transition to a ready queue strictly after all their predecessors have successfully completed. The scheduler subsequently prioritizes these ready tasks for sequential resource allocation. However, simultaneously scheduling multiple workflows naturally leads to heavy resource competition. The consolidation of tasks onto high-performance nodes reduces global power consumption but frequently induces severe queuing delays that lead to workflow deadlines violations. To reconcile this conflict, this study formulates a model to optimize energy efficiency under deadline constraints.

TABLE I
Key Notations and Their Definitions

Notation	Description
$\mathcal{G}(\tau)$	the set of workflows that have arrived by time τ
$\mathcal{T}$	a set of tasks
$\mathcal{V}$	a set of VMs
$\mathcal{H}$	a set of hosts
$\mathcal{R}_\tau$	the ready-task set at time τ
$W(\tau)$	the number of workflows that have arrived by time τ
G_w	the w-th workflow
t_i	the i-th task
v_j	the j-th VM
h_k	the k-th host
n	the number of tasks
m	the number of VMs
K	the number of hosts
l_i	the length of task t_i
F_i^{in}	the input files of task t_i
$F_{e,i}^{in}$	the external input files of task t_i
$S_{i',i}^{in}$	the size of input files from task $t_{i'}$ to task t_i
$S_{e,i}^{in}$	the size of external input files to task t_i
F_i^{out}	the output files of task t_i
S_i^{out}	the size of output files of task t_i
$\mathrm{pre}(\cdot)$	the set of predecessors of a task
τ_i^{in}	the time required to read the input file of task t_i
τ_i^{out}	the time required to write the output file of task t_i
τ_i^{exe}	the execution time of task t_i
τ_i^r	the ready time of task t_i
τ_i^w	the waiting time of task t_i
τ_i^s	the start time of task t_i
τ_i^e	the end time of task t_i
τ_i^d	the deadline of task t_i
τ_i^a	the available time of VM for task t_i
$\boldsymbol{T}_w^{arr}$	the arrival time of workflow G_w
$\boldsymbol{T}_w^d$	the deadline of workflow G_w
$\boldsymbol{T}_w^e$	the completion time of workflow G_w
M_w	the makespan of workflow G_w
M_w^{base}	heuristic baseline makespan of workflow G_w
α_d	deadline stringency factor
C_k^h	the processing capacity of host h_k
C_j^v	the processing capacity of VM v_j
L_τ^k	the load of host h_k at time τ
$\mathrm{power}_k(\cdot)$	the load-power function of host h_k
$E_k^{[\tau_1,\tau_2]}$	the energy consumed by host h_k in period $[\tau_1, \tau_2]$

B. Mathematical Formulation

This study considers a dynamic online workflow scheduling problem with continuous, unpredictable arrivals. Key notations are summarized in Table I. Let $W(\tau)$ denote the arrival counting process up to time τ. The set of arrived workflows by time τ is defined as

$$\mathcal{G}(\tau) = \{G_w \mid \boldsymbol{T}_w^{arr} \leq \tau\}. \tag{1}$$

For stochastic arrivals, $W(\tau)$ can be modeled as a Poisson counting process with rate λ, i.e.,

$$W(\tau + \Delta\tau) - W(\tau) \sim \mathrm{Poisson}(\lambda \Delta\tau). \tag{2}$$

Within a workflow $G_w = (\mathcal{T}_w, \mathcal{E}_w)$, non-entry tasks receive inputs from their predecessors, while entry tasks receive external inputs. After execution, tasks produce output files forwarded to their child tasks. Let S_f denote the size of file f. The inter-task input data $S^{in}_{i',i}$ from parent $t_{i'}$ to child t_i, the external input data $S^{in}_{e,i}$ for entry task t_i, and the total output data S^{out}_i of task t_i are computed as:

$$S^{in}_{i',i} = \sum_{f \in F^{in}_i \cap F^{out}_{i'}} S_f, \quad S^{in}_{e,i} = \sum_{f \in F^{in}_{e,i}} S_f \tag{3}$$

$$S^{out}_i = \sum_{f \in F^{out}_i} S_f. \tag{4}$$

Then, τ^{in}_i accounts for external and parent-to-child transfers, with external input omitted for non-entry tasks and parent transfer omitted for entry tasks, while τ^{out}_i depends on the output data size and the assigned VM bandwidth. Inter-task transmission is considered negligible when parent and child share the same resource. After converting data sizes from bytes to bits, τ^{in}_i and τ^{out}_i are given by Equation (5), where b_i is the allocated VM bandwidth.

$$\tau^{in}_i = 8 \cdot \frac{S^{in}_{e,i}}{b_i} + \sum_{t_{i'} \in \mathrm{pre}(t_i)} 8 \cdot \frac{S^{in}_{i',i}}{b_i}, \quad \tau^{out}_i = 8 \cdot \frac{S^{out}_i}{b_i} \tag{5}$$

Accordingly, the execution of task t_i consists of three stages, namely input transmission, computation, and output transmission. Its overall execution duration τ^{exe}_i is defined as

$$\tau^{exe}_i = \tau^{in}_i + \frac{l_i}{c_i} + \tau^{out}_i, \tag{6}$$

where c_i represents the processing capacity of the VM assigned to task t_i and l_i denotes the workload of task t_i.

In dynamic execution, a task becomes ready only after all its predecessors have completed execution. Therefore, the ready time τ^r_i of task t_i is given by

$$\tau^r_i = \begin{cases} \boldsymbol{T}^{arr}_w, & \mathrm{pre}(t_i) = \varnothing, \\ \max\limits_{t_{i'} \in \mathrm{pre}(t_i)} \tau^e_{i'}, & \text{otherwise.} \end{cases} \tag{7}$$

At any time τ, only tasks of already arrived workflows are considered for scheduling. A task can enter the ready queue only if it remains unscheduled and all its precedence constraints have been satisfied. Accordingly, the ready-task set is defined as

$$\mathcal{R}_\tau = \{ t_i \in \mathcal{U}_\tau \mid \tau^r_i \le \tau \}, \tag{8}$$

where $\mathcal{U}_\tau$ denotes the set of all unscheduled tasks within $\mathcal{G}(\tau)$. Tasks in $\mathcal{R}_\tau$ are then prioritized and sequentially allocated to available VMs. Thus, the start time, waiting time, and completion time of task t_i are respectively determined by:

$$\tau^s_i = \max\{\tau^r_i, \tau^a_i\}, \quad \tau^w_i = \tau^s_i - \tau^r_i, \quad \tau^e_i = \tau^s_i + \tau^{exe}_i. \tag{9}$$

At the workflow level, the completion time $\boldsymbol{T}^e_w$ of G_w is the maximum completion time among its exit tasks $\mathcal{T}^{exit}_w$:

$$\boldsymbol{T}^e_w = \max_{t_i \in \mathcal{T}^{exit}_w} \tau^e_i. \tag{10}$$

Therefore, the makespan of workflow G_w is

$$M_w = \boldsymbol{T}^e_w - \boldsymbol{T}^{arr}_w. \tag{11}$$

For each evaluation episode, let $\mathcal{G}_{eval}$ denote the set of workflows evaluated in that episode, with $|\mathcal{G}_{eval}| = W_{eval}$. The deadline success rate (SR) is defined as the fraction of workflows completed within their absolute deadlines T^d_w:

$$SR = \frac{1}{W_{eval}} \sum_{G_w \in \mathcal{G}_{eval}} \mathbb{I}\Big(\boldsymbol{T}^e_w \le \boldsymbol{T}^d_w\Big), \tag{12}$$

where $\mathbb{I}(\cdot)$ is the indicator function that returns 1 if the condition is true and 0 otherwise.

At the host level, the instantaneous load of host h_k at time τ is the sum of the capacity ratios of its active VMs:

$$L^k_\tau = \sum_{t_i \in \mathcal{T}} \sum_{v_j \in \mathcal{V}_k} \frac{C^v_j}{C^h_k} \cdot x_{i,j,\tau}, \tag{13}$$

where $x_{i,j,\tau} \in \{0,1\}$ indicates whether task t_i runs on VM v_j at time τ, $\mathcal{V}_k$ is the set of VMs on host h_k, and C^v_j and C^h_k denote the VM and host processing capacities, respectively.

The energy consumed by host h_k over an arbitrary interval $[\tau_1, \tau_2]$ is obtained by integrating its load-dependent power consumption function [2], which is derived from the SPECpower benchmark[1]:

$$E^{[\tau_1,\tau_2]}_k = \int_{\tau_1}^{\tau_2} \mathrm{power}_k\big(L^k_\tau\big)\, d\tau. \tag{14}$$

At the evaluation level, the scheduling target is to minimize the total episode energy, while deadline satisfaction is assessed through a probabilistic QoS constraint:

$$\min_{\pi} \quad \sum_{k=1}^{K} \int_{\min_{G_w \in \mathcal{G}_{eval}} \boldsymbol{T}^{arr}_w}^{\max_{G_w \in \mathcal{G}_{eval}} \boldsymbol{T}^e_w} \mathrm{power}_k\big(L^k_\tau\big)\, d\tau \tag{15}$$

$$\text{s.t.} \quad \mathbb{P}(SR \ge \Gamma) \ge \beta. \tag{16}$$

IV. The Proposed Algorithm

A. Overview

By exploiting the decentralized and adaptive mechanisms of multi-agent systems, MAHDRL facilitates the efficient resolution of the DECWSP-DC. Accordingly, this problem is modeled as a partially observable MDP [3] defined by the tuple $\mathcal{M} = \langle N, \mathcal{S}, \mathcal{A}, P, R, \Omega, O, \gamma \rangle$, where:

- N: the number of agents.
- $\mathcal{S}$: a finite set of global states.
- $\mathcal{A}$: the joint action space, $\mathcal{A} = \prod_{i=1}^{N} \mathcal{A}_i$, with $\mathcal{A}_i$ being the action set of agent i.
- P: $P(s' \mid s, \mathbf{a})$, the state transition function specifying the probability of reaching s' given current state s and a joint action $\mathbf{a}$ within the discrete space $\mathcal{A}$.
- Ω: the joint observation space, $\Omega = \prod_{i=1}^{N} \Omega_i$, with Ω_i being the observation set of agent i.
- O: $O(\mathbf{o}_t \mid s, \mathbf{a})$, the observation probability of receiving joint observation $\mathbf{o}_t \in \Omega$ given state s and joint action $\mathbf{a}$.
- R: $R(s, \mathbf{a})$, the reward function yielding a scalar reward for executing joint action $\mathbf{a}$ in state s.
- $\gamma \in (0, 1]$: the discount factor.

Each agent i independently determines its action a^i according to its policy $\pi_{\theta_i}(a^i \mid o^i)$.

[1] http://www.spec.org/power_ssj2008/results/power_ssj2008.html

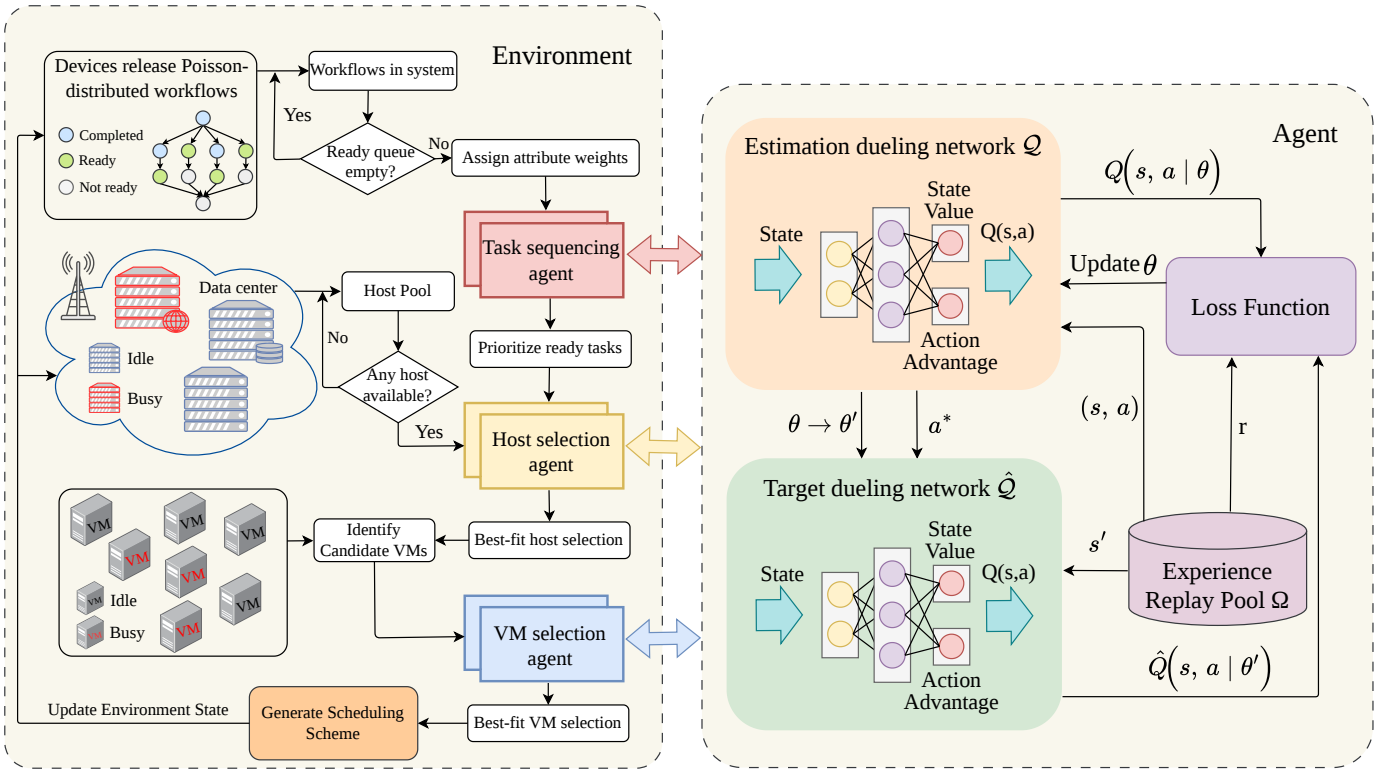


Fig. 2. The hierarchical architecture of the proposed MAHDRL framework.

As depicted in Fig. 2, MAHDRL addresses dynamic scheduling complexity through a coordinated three-tier architecture comprising three specialized agents: the task sequencing agent (TA), host selection agent (HA), and VM selection agent (VA). This hierarchy decouples the scheduling logic, with TA prioritizing tasks, HA selecting target hosts, and VA mapping tasks to specific VMs, thereby reducing the decision space. The corresponding network architectures and training procedures are detailed in the following subsections.

B. Task Sequencing Agent

As the highest-level entity in MAHDRL, the TA adaptively adjusts heuristic weights to prioritize ready tasks, guiding downstream allocation for phase-level energy optimization under strict deadline constraints.

1) State Space: The decision-making process of the TA is modeled as an event-driven sequential decision problem operating over discrete phase steps τ_p. Activated by critical state transitions, such as the arrival of new workflows or dynamic shifts in queue status, the TA employs a partial observation strategy to effectively mitigate intractable complexity inherent in the state space. Specifically, by filtering out unready tasks and low-level resource fluctuations, this strategy shields the TA from short-term scheduling noise, enabling it to efficiently learn stable, system-wide sequencing policies. To formalize this approach, the TA extracts essential high-level environmental features at the current decision step τ_p to construct a concise observation vector $\mathbf{o}_{\tau_p}^{TA} \in \Omega_{TA}$, which is logically partitioned into four distinct categories:

Task Queue Statistics ($\mathbf{B}_{que}$): Quantifying the backlog and waiting severity of unscheduled tasks. The task ready (RR_{τ_p}) and unready (URR_{τ_p}) rates are formulated as:

$$RR_{\tau_p} = \min\left(1, \frac{|\mathcal{R}_{\tau_p}|}{\max(1, n_r^{max})}\right) \tag{17}$$

$$URR_{\tau_p} = 1 - RR_{\tau_p} \tag{18}$$

where n_r^{max} is the historical maximum size of $\mathcal{R}_{\tau_p}$. At the current time τ_{now}, the average (AWT_{τ_p}) and maximum (MWT_{τ_p}) waiting times of ready tasks are computed as:

$$AWT_{\tau_p} = \frac{1}{|\mathcal{R}_{\tau_p}|} \sum_{t_i \in \mathcal{R}_{\tau_p}} (\tau_{now} - \tau_i^r) \tag{19}$$

$$MWT_{\tau_p} = \max_{t_i \in \mathcal{R}_{\tau_p}} (\tau_{now} - \tau_i^r) \tag{20}$$

Global Resource Utilization ($\mathbf{B}_{res}$): These metrics reflect the aggregate workload and availability of the underlying infrastructure. RUR_{τ_p} and RIR_{τ_p} represent the overall VM utilization and idle ratios:

$$RUR_{\tau_p} = \frac{|\mathcal{V}_{busy}|}{|\mathcal{V}|}, \quad RIR_{\tau_p} = 1 - RUR_{\tau_p} \tag{21}$$

where $\mathcal{V}_{busy}$ represents the set of busy VMs. At the physical machine level, HLD_{τ_p} and HRR_{τ_p} capture the mean host computational load and running VM ratio:

$$HLD_{\tau_p} = \frac{1}{K} \sum_{k=1}^{K} L_{\tau_p}^k, \quad HRR_{\tau_p} = \frac{1}{K} \sum_{k=1}^{K} \frac{|\mathcal{V}_{busy}^k|}{|\mathcal{V}_{h_k}|} \tag{22}$$

where $\mathcal{V}_{busy}^{k}$ and $\mathcal{V}_{h_k}$ denote the set of currently active virtual machines and the set of total virtual machines deployed on host k, respectively.

Deadline Urgency Metrics ($\mathbf{B}_{urg}$): This subset evaluates the temporal criticality of the ready tasks. MSL_{τ_p} and ASL_{τ_p} evaluate the minimum and average workflow slack among ready tasks, normalized by the global maximum deadline $\boldsymbol{T}_{\max}^{d} = \max_{G_w \in \mathcal{G}} \boldsymbol{T}_{w}^{d}$:

$$MSL_{\tau_p} = \frac{\min\limits_{t_i \in \mathcal{R}_{\tau_p}} \left(\boldsymbol{T}_{w(i)}^{d} - \tau_{now} - \hat{\tau}_i \right)}{\boldsymbol{T}_{\max}^{d}} \tag{23}$$

$$ASL_{\tau_p} = \frac{1}{|\mathcal{R}_{\tau_p}|} \sum_{t_i \in \mathcal{R}_{\tau_p}} \frac{\boldsymbol{T}_{w(i)}^{d} - \tau_{now} - \hat{\tau}_i}{\boldsymbol{T}_{\max}^{d}} \tag{24}$$

where $\hat{\tau}_i$ defines the estimated execution time based on average VM capacities, and $\boldsymbol{T}_{w(i)}^{d}$ denotes the absolute deadline of the workflow to which task t_i belongs.

Heuristic Context ($\mathbf{B}_{heu}$): $\mathcal{F}$ represents the current 5-dimensional heuristic weights applied in the sequencing strategy, as detailed in IV-B2.

2) Action Space: At each decision point τ_p, the TA dynamically adjusts task sequencing weights via a feature set $\mathcal{F}$, mapping arrival time, workload, critical path length, slack time, and deadlines to the first-come first-served, shortest job first, most critical first, highest urgency ratio first, and earliest deadline first heuristics. To ensure a consistent priority direction, these features are min-max normalized to $[0, 1]$ relative to R_{τ_p} via ascending or descending scaling, ensuring higher values consistently denote higher scheduling priorities.

The discrete action space $\mathcal{A}^{TA}$ provides 3^5 options where each action $a_{\tau_p}^{TA}$ updates the weight $\mathbf{w}_{\tau_p}$ via an increment vector $\Delta\mathbf{w}_{\tau_p} \in \{-\delta, 0, +\delta\}^5$. An action mask $\mathbf{M}_{\tau_p}^{TA}$ subsequently filters invalid increments to enforce non-negativity and normalization constraints for feasible policy exploration.

3) Sequencing Mechanism and Hierarchical Coordination: Leveraging the updated weights $\mathbf{w}_{\tau_p}$, the system computes the priority score for each task $t_i \in \mathcal{R}_{\tau_p}$ via a linear weighted summation of the five normalized features $\hat{f}_k(t_i)$:

$$\text{score}(t_i) = \sum_{k \in \mathcal{F}} w_k \cdot \hat{f}_k(t_i) \tag{25}$$

The resulting ordered task queue serves as an ordered input for the subsequent scheduling phase. Specifically, the lower-level agents (HA and VA) sequentially adhere to this order to execute fine-grained resource mapping. This mechanism coordinates task sequencing with resource allocation.

4) Reward Function: To align the TA with global multi-objective optimization, a stage-level composite reward $r_{\tau_p}^{TA}$ is designed. This reward penalizes both phase-level energy consumption and task deadline tardiness, ensuring stringent QoS compliance:

$$r_{\tau_p}^{TA} = -\left[\alpha_t \cdot \frac{\Delta\text{Delay}_{\tau_p}}{D_{ref}} + (1 - \alpha_t) \cdot (\omega_e \cdot \Delta E_{\tau_p}) \right] \tag{26}$$

where ΔE_{τ_p} and $\Delta\text{Delay}_{\tau_p}$ denote the energy increase and accumulated tardiness in the current phase, respectively. This reward serves as a training surrogate for the energy-minimization objective, where deadline violations are converted into tardiness penalties. The parameters D_{ref} and ω_e act as global scaling factors, harmonizing the magnitudes of time and energy across the entire MAHDRL framework. $\alpha_t \in [0, 1]$ modulates the TA's preference between delay and energy.

C. Host Selection Agent

As the intermediate layer, the HA matches dequeued tasks to physical hosts based on system context, balancing utilization and reducing downstream action space to alleviate joint mapping complexity.

1) State Space: Unlike the phase-based updates of the TA, the HA executes its routing policy in a continuous, event-driven manner. Its decision step τ_h is triggered when a ready task is dequeued for host assignment. At this moment, the HA employs a partial observation strategy to handle the stochasticity of future arrivals and low-level execution fluctuations. To guarantee computational efficiency while providing a comprehensive routing context, its observation vector $\mathbf{o}_{\tau_h}^{HA} \in \Omega_{HA}$ is constructed by concatenating three distinct information blocks:

Current Task Features ($\mathbf{B}_{task}$): This 12-dimensional vector captures the heterogeneous attributes and structural dependencies of the pending task t_i. It integrates intrinsic characteristics (normalized computational workload $\tilde{l}_i$, I/O data sizes, and I/O ratio), topological DAG influences (normalized child count, upward rank, and downward rank), the normalized waiting time, and the proportion of same-workflow ready tasks to capture parallel clustering effects. Furthermore, letting $\hat{\tau}_{i,v}$ denote the estimated execution time of t_i on an idle $VM_v \in \mathcal{V}_{idle}$, the optimal slack time (OSL_i) is formulated as:

$$OSL_i = \frac{\boldsymbol{T}_{w(i)}^{d} - (\tau_{now} + \min\limits_{v \in \mathcal{V}_{idle}} \hat{\tau}_{i,v})}{\boldsymbol{T}_{\max}^{d}} \tag{27}$$

This block also includes the similarly normalized optimal and average execution time estimates.

Global System Features ($\mathbf{B}_{sys}$): This 10-dimensional vector provides the system-wide scheduling context. To maintain hierarchical consistency, the HA directly inherits seven core metrics $(RR, URR, RUR, AWT, MWT, MSL, ASL)$ from the TA observation space. The remaining three dimensions characterize the aggregate state of the current ready queue $\mathcal{R}_{\tau_h}$. Formally, the normalized total computational workload (NTW_{τ_h}) is computed as:

$$NTW_{\tau_h} = \frac{\sum_{t_i \in \mathcal{R}\tau_h} \hat{l}_i}{\bar{l}} \tag{28}$$

Additionally, this block incorporates the urgent task proportion $UTP\tau_h = \frac{|\mathcal{R}_{urgent}|}{|\mathcal{R}_{\tau_h}|}$, where $\mathcal{R}_{urgent}$ isolates tasks with an estimated slack below zero, and the workflow distribution breadth $WDB_{\tau_h} = \frac{|\mathcal{G}_{ready}|}{|\mathcal{G}|}$.

Candidate Host Features ($\mathbf{B}_{host}$): The HA constructs a 5-dimensional profile for each of the K candidate hosts, expanding this block to $5K$ dimensions. For a given host h_k, its profile integrates the normalized total processing capacity, the active computational load ratio, the proportions of idle and running VMs, and the normalized earliest available latency (EAL_k). Formally, EAL_k evaluates the minimum temporal

Algorithm 1: Training Procedure of MAHDRL

Input: $\mathcal{G}, \mathcal{H}, \mathcal{V}, \eta, \gamma, \rho$, batch size B, max episodes I_{max}
Output: Trained network parameters $\theta_{TA}, \theta_{HA}, \theta_{VA}$.

Init. $\mathcal{D}_x \leftarrow \emptyset$ and $\theta_x \sim random\ \forall x \in \{TA, HA, VA\}$;
Init. $\theta'_x \leftarrow \theta_x\ \forall x \in \{TA, HA, VA\}$;
for episode $= 1$ **to** I_{max} **do**
  Init. environment Env, set clock $\tau_{now} \leftarrow 0$;
  Inject workflows $W \sim \text{Poisson}(\lambda)$ into Env;
  while Env is not in a terminal state **do**
    Execute Algorithm 2 to interact with Env and collect experiences into $\mathcal{D}_x$;
    foreach agent $x \in \{TA, HA, VA\}$ **do**
      if $|\mathcal{D}_x| \geq B$ and $\text{UpdateReady}(x)$ **then**
        // $e_i = (s_i, m_i, a_i, r_i, s'_i, m'_i, d_i)$
        $\mathcal{B}_x = \{e_i\}_{i=1}^{B} \sim \mathcal{D}_x$;
        foreach $e_i \in \mathcal{B}_x$ **do**
          if $d_i = 1$ or $\sum_{a'} m'_i(a') = 0$ **then**
            $y_i \leftarrow r_i$;
          else
            $a^* \leftarrow \arg\max_{a'} Q_{\theta_x}(s'_i, a')$, subject to $m'_i(a') = 1$;
            $y_i \leftarrow r_i + \gamma Q_{\theta'_x}(s'_i, a^*)$;
        $L(\theta_x) = \frac{1}{B} \sum_{i=1}^{B} (y_i - Q_{\theta_x}(s_i, a_i))^2$;
        $\theta_x \leftarrow \theta_x - \eta \nabla_{\theta_x} L(\theta_x)$;
        $\theta'_x \leftarrow \rho_x \theta_x + (1 - \rho_x)\theta'_x$;
return $\theta_{TA}, \theta_{HA}, \theta_{VA}$;

Algorithm 2: Hierarchical Scheduling Execution

Input: Env, $\mathcal{G}$, $\theta_{TA}, \theta_{HA}, \theta_{VA}$.
Output: Updated Env and $\mathcal{D}_{TA}, \mathcal{D}_{HA}, \mathcal{D}_{VA}$.

$\mathcal{R} \leftarrow \text{ExtractReadyTasks}(\mathcal{G})$; is_TA_trig $\leftarrow$ False;
if $\mathcal{R} \neq \emptyset \wedge \text{TriggerCond}(TA)$ **then**
  $\mathbf{o}^{TA} \leftarrow [\mathbf{B}_{que}, \mathbf{B}_{res}, \mathbf{B}_{urg}, \mathbf{B}_{heu}]$;
  Get mask $\mathbf{M}^{TA}$ for valid weight increments;
  Select $a^{TA} = \pi^{\epsilon}_{\theta_{TA}}(\mathbf{o}^{TA} \mid \mathbf{M}^{TA})$;
  Update weights $\mathbf{w} \leftarrow \text{Norm}(\mathbf{w} + \Delta\mathbf{w}(a^{TA}))$;
  $\mathcal{R} \leftarrow \text{Sort}(\mathcal{R}, \mathbf{w})$; is_TA_trig $\leftarrow$ True;
foreach $t_i \in \mathcal{R}$ **do**
  if no idle VM exists **then**
    break;
  $\mathbf{o}^{HA}_{\tau_h} \leftarrow [\mathbf{B}_{task}, \mathbf{B}_{sys}, \mathbf{B}_{host}]$;
  Get mask $\mathbf{M}^{HA}$ for hosts with idle VMs;
  Select host $h_k = \pi^{\epsilon}_{\theta_{HA}}(\mathbf{o}^{HA}_{\tau_h} \mid \mathbf{M}^{HA})$;
  $\mathbf{o}^{VA}_{\tau_v} \leftarrow [\mathbf{o}^{HA}_{\tau_h}, \mathbf{B}_{vm}]$;
  Get mask $\mathbf{M}^{VA}$ for valid idle VM slots;
  Select VM $v = \pi^{\epsilon}_{\theta_{VA}}(\mathbf{o}^{VA}_{\tau_v} \mid \mathbf{M}^{VA})$;
  Map $t_i \mapsto v$, step Env, get latency and ΔE;
  Observe rewards $r^{HA}_{\tau_h}, r^{VA}_{\tau_v}$, next states $\mathbf{o}'$, masks $\mathbf{M}'$, flags d;
  Store $(\mathbf{o}, \mathbf{M}, a, r, \mathbf{o}', \mathbf{M}', d)^{HA} \rightarrow \mathcal{D}_{HA}$;
  Store $(\mathbf{o}, \mathbf{M}, a, r, \mathbf{o}', \mathbf{M}', d)^{VA} \rightarrow \mathcal{D}_{VA}$;
if is_TA_trig == True **and** $PhaseTerminates()$ **then**
  Compute delayed reward r^{TA} via phase metrics;
  Observe next state $\mathbf{o}'^{TA}$, mask $\mathbf{M}'^{TA}$, flag d^{TA};
  Store $(\mathbf{o}, \mathbf{M}, a, r, \mathbf{o}', \mathbf{M}', d)^{TA} \rightarrow \mathcal{D}_{TA}$;
return Updated Env,$\mathcal{D}_{TA}, \mathcal{D}_{HA}, \mathcal{D}_{VA}$;

delay τ^a before any VM within h_k becomes available. To ensure dimensional consistency, this raw delay is divided by the global maximum deadline:

$$EAL_k = \frac{\min\limits_{v \in h_k} \max(0, \tau^a - \tau_{now})}{\boldsymbol{T}^d_{\max}} \tag{29}$$

2) Action Space: The action space of the HA is defined as a discrete set $\mathcal{A}^{HA} = \{0, 1, \ldots, K-1\}$, assigning the pending task to one of the K physical hosts. To guarantee allocation validity, a dynamic action mask restricts the agent to sample exclusively from a feasible subset of hosts possessing idle VMs. This pruning mechanism precludes invalid assignments, reduces the exploration space, and expedites training convergence.

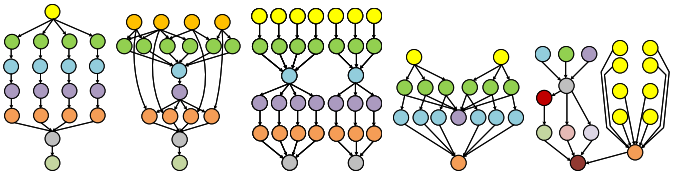
3) Reward Function: The immediate reward $r^{HA}_{\tau_h}$ balances the global energy increment with task-level tardiness to penalize sub-deadline violations. The energy cost is the system energy increase ΔE_{τ_h} scaled by ω_e. Meanwhile, task tardiness is evaluated as $\max(0, \tau^e_i - \tau^d_i)$, where the sub-deadline τ^d_i is obtained by distributing the global deadline backward through the DAG. Starting from the final tasks, each sub-deadline is calculated by subtracting the average processing and communication times from the deadline of the next task. This tardiness is normalized by D_{ref} and limited to $[0, 1]$ to keep training stable. Modulated by a weighting coefficient α_h, the final reward is defined as follows:

$$\begin{aligned} r^{HA}_{\tau_h} = & -(1 - \alpha_h) \cdot (\omega_e \cdot \Delta E_{\tau_h}) \\ & - \alpha_h \cdot \text{clip}\left(\frac{\max(0, \tau^e_i - \tau^d_i)}{D_{ref}}, 0, 1\right) \end{aligned} \tag{30}$$

D. Virtual Machine Selection Agent

As the final decision tier, the VA maps host-allocated tasks to designated virtual machines based on local hardware to directly minimize dynamic energy and satisfy strict deadlines.

1) State Space: The VA observation vector $\mathbf{o}^{VA}_{\tau_v} \in \Omega_{VA}$ is formed by four information blocks. It inherits $\mathbf{B}_{task}$ and $\mathbf{B}_{sys}$ from the HA to retain task-level and global context, incorporates $\mathbf{B}_{host}$ to describe the selected host h_k, and adds the VM-level block $\mathbf{B}_{vm}$. Spanning $5V_{max}$ dimensions, $\mathbf{B}_{vm}$ contains five metrics for each candidate VM slot including idle status, normalized availability delay, relative processing capacity, normalized bandwidth, and predicted task finish time. Thus, the total observation dimension is $\dim(\mathbf{o}^{VA}_{\tau_v}) = 12 + 10 + 5 + 5V_{max} = 27 + 5V_{max}$.



(a) Epig (b) Montage (c) LIGO (d) Cyber (e) SIPHT

Fig. 3. Structures of workflows (Epig: Epigenomics, Cyber: CyberShake).

2) Action Space: For VM-level mapping, each action selects a VM slot within the designated host. Formally, the action set is defined as $\mathcal{A}^{VA} = \{0, 1, \ldots, V_{max} - 1\}$, where V_{max} is the maximum number of VMs on any host. A dynamic action mask ensures allocation validity by allowing only idle and physically existent VM slots, thereby excluding occupied VMs and padded invalid dimensions.

3) Reward Function: The immediate reward $r^{VA}_{\tau_v}$ adopts a penalty structure analogous to the HA's, but shifts the energy focus to the designated host h_k. The localized energy penalty quantifies the actual energy increment ΔE_{h_k} scaled by ω_e. Simultaneously, the normalized and clipped task-level tardiness penalty is retained. Modulated by a trade-off coefficient α_v, the final VA reward is formulated as:

$$\begin{aligned} r^{VA}_{\tau_v} = & -(1-\alpha_v)\cdot(\omega_e \cdot \Delta E_{h_k}) \\ & -\alpha_v \cdot \text{clip}\left(\frac{\max(0, \tau^e_i - \tau^d_i)}{D_{ref}}, 0, 1\right) \end{aligned} \tag{31}$$

To ensure optimization consistency across global and local scheduling levels, the shared scaling parameters are empirically set to $\omega_e = 10^{-3}$ and $D_{ref} = 300$. Furthermore, a uniform trade-off coefficient of $\alpha_t = \alpha_h = \alpha_v = 0.75$ is applied across all agents, unifying the energy-delay preference throughout the MAHDRL framework.

E. Overall Training Procedure

Algorithm 1 outlines the optimization and training procedure of MAHDRL. During each episode, agents interact with the environment following Algorithm 2, and the resulting transitions and rewards are stored in their respective replay buffers denoted by $\mathcal{D}_{TA}$, $\mathcal{D}_{HA}$, and $\mathcal{D}_{VA}$ to update the dueling networks, which decouple state-value and action-advantage estimation. Each agent is updated independently when $UpdateReady(x)$ holds, i.e., when sufficient samples are available and the scheduled update frequency is reached. Specifically, HA and VA are updated after task assignments, whereas TA is updated only after a sequencing phase terminates. For each sampled mini-batch $\mathcal{B}_x$, temporal-difference targets are computed to minimize the mean squared error loss and update θ_x with learning rate η, followed by a soft target-network update via Polyak averaging with coefficient ρ_x.

V. Computational Experiments

This section evaluates the scheduling performance of MAHDRL through extensive simulations. All experiments were implemented in Python 3.12 and conducted on a Windows workstation equipped with an Intel Core i7-14700KF CPU at 3.40 GHz, 32 GB of RAM, and an NVIDIA GeForce RTX 4060 Ti GPU with 8 GB of VRAM.

TABLE II
The Types of Workflows

Type	Small(S)	Medium(M)	Large(L)
CyberShake	30	50	100
Epigenomics	24	47	100
Ligo	30	50	100
Montage	25	50	100
Sipht	25	58	97

TABLE III
Scenarios of Different Scales.

Scenario	Workflow Composition	Resources Scales	Host Number	VM Number
SS	S	S	3	(9, 8, 8)
SM	S	M	6	(9, 9, 8, 8, 8, 8)
SL	S	L	9	(9, 9, 9, 8, 8, 8, 8, 8, 8)
MS	S + M	S	3	(9, 8, 8)
MM	S + M	M	6	(9, 9, 8, 8, 8, 8)
ML	S + M	L	9	(9, 9, 9, 8, 8, 8, 8, 8, 8)
LS	S + M + L	S	3	(9, 8, 8)
LM	S + M + L	M	6	(9, 9, 8, 8, 8, 8)
LL	S + M + L	L	9	(9, 9, 9, 8, 8, 8, 8, 8, 8)

A. Simulation Setup

1) Problem Instances: For a consistent evaluation, this study utilizes five scientific workflows (CyberShake, Epigenomics, LIGO, Montage, SIPHT) [37], [38] across three scales outlined in Table II, preserving their original topologies depicted in Fig. 3. Task workloads are sampled from [60k, 90k] MI, with 1-3 input/output files (512-1024 MB each) per task. Dynamic workflow arrivals follow a Poisson distribution [39] with rate λ.

Infrastructure heterogeneity is simulated using three physical host types: hty1, hty2, and hty3, whose energy profiles are modeled upon SPECpower benchmark data. Based on this heterogeneous setup, crossing the three workflow and cluster resource scales yields nine distinct evaluation scenarios as detailed in Table III. For clarity, these two-letter identifiers directly denote the paired scales, such as SS representing a small workflow scale on a small resource pool. For VM provisioning, processing capacity (PC) and bandwidth (BW) are paired into five discrete tiers: PC $\in \{1k, 2k, 4k, 6k, 8k\}$ MI/s and BW $\in \{1k, 2k, 4k, 6k, 8k\}$ Mb/s. These tiers are cyclically assigned to local VMs via a round-robin strategy to ensure intra-host resource balance.

Regarding temporal constraints, workflow deadlines are derived from baseline makespans following [40]. For each workflow G_w, the baseline makespan M^{base}_w is obtained using the earliest-ready-time-first with first-idle-first-serve (ERTF-FIFS) heuristic, which combines ERTF task ordering with earliest-idle VM assignment. The absolute deadline is then defined as

$$\boldsymbol{T}^d_w = \boldsymbol{T}^{arr}_w + \alpha_d \cdot M^{base}_w, \tag{32}$$

TABLE IV
SYSTEM AND TRAINING HYPERPARAMETERS

Parameter	Setting (TA / HA, VA)
Workflows per episode W_{eval}	50
Penalty factor δ	0.1
Batch size B	256
Max episodes I_{max}	600
Arrival rate λ	0.03
Discount factor γ	0.95
Learning rate η	3×10^{-4}
Polyak coefficient ρ_x	0.01 / 0.005
Buffer size	$6 \times 10^4 / 1.2 \times 10^5$
ϵ-decay steps	$4 \times 10^4 / 8 \times 10^4$
Hidden layers	$512^2, 256, 128 / 1024^2, 512^2, 256$

where T_w^{arr} denotes the arrival time of G_w, and α_d is the deadline stringency factor. Rather than using a fixed α_d, each workflow independently samples α_d from $\{2, 3\}$ to form three deadline tightness levels. Specifically, the sampling probabilities over $\{2, 3\}$ are $(0.8, 0.2)$, $(0.5, 0.5)$, and $(0.2, 0.8)$ for the Tight, Medium, and Loose levels, respectively. Finally, to ensure experimental reproducibility, the key hyperparameters and training configurations are summarized in Table IV.

2) Comparative Algorithms: The proposed MAHDRL is benchmarked against three state-of-the-art DRL frameworks and two heuristic baseline. For fairness, each DRL baseline retains its original decision structure and learning architecture, while its input features and reward terms are adapted only to align with the common energy-tardiness objective and its own action granularity. Furthermore, these flat baselines exclude any hierarchical decision variables unique to MAHDRL to preserve the integrity of the structural comparison. The selected algorithms are described as follows.

- **PD3QN** [12] employs the ERTF heuristic for task sequencing, while a D3QN agent outputs optimal allocations from idle VMs.
- **MARL** [3] also utilizes ERTF for sequencing. Its resource allocation is managed by a two-level advantage actor-critic agent that incorporates action masking while hierarchically selecting a target host and an idle VM.
- **IRWS** [13] replaces heuristic sequencing with a lightweight feed-forward neural network to score ready tasks. A PPO agent handles VM allocation, achieving joint optimization.
- **ERTF-H-FIFS** is a hierarchical variant of the ERTF-FIFS baseline. While retaining ERTF for task sequencing, it upgrades the resource allocation into a two-stage cascaded FIFS strategy, greedily selecting the earliest available host before subsequently mapping the task to the earliest idle VM within it.

For statistical robustness, all algorithms are evaluated over 30 runs across 27 scenarios comprising three stringency levels and nine base configurations. These evaluations utilize identical workflow arrivals, workload and I/O samples, VM/host configurations, deadline baselines, and random seeds. The DRL methods are trained under comparable episode budgets and evaluation protocols, ensuring that observed differences mainly arise from scheduling policies rather than instance generation or resource settings.

B. Comparison with State-of-the-Art Algorithms

This section evaluates the MAHDRL framework through a multi-dimensional analysis of energy efficiency, deadline reliability, and statistical significance across across various heterogeneous composite scenarios.

Regarding energy optimization and robustness, Table V and Fig. 4 demonstrate that MAHDRL achieves the best or highly competitive energy performance in most scenarios. Its energy distributions generally remain at relatively low levels with narrow interquartile ranges and fewer outliers, indicating stable performance. While MARL achieves marginally lower energy in specific unbalanced cases (e.g., MS and LS) while maintaining a high ESP, this suggests that MAHDRL's hierarchical sequencing may introduce additional routing conservativeness under certain resource-contention patterns. Unlike non-hierarchical baselines (PD3QN, IRWS) and partially hierarchical resource allocation of MARL, MAHDRL comprehensively decouples decisions across the dynamic task, host, and VM levels. This three-tier architecture effectively manages large decision spaces, reducing the risk of poor local decisions and contributing to competitive overall performance.

Consistent with the QoS criterion in Equation (16), Table V evaluates reliability through the empirical satisfaction probability (ESP), which denotes the proportion of independent runs satisfying $SR \geq \Gamma$. Under configurations defined by $\Gamma = 0.92$ and $\beta = 0.99$, while most baselines comply, the ESP for IRWS drops to 20% in the Tight MS scenario, revealing a poor trade-off between energy and reliability. Conversely, MAHDRL maintains 100% ESP across all configurations, confirming that its penalized policy effectively balances energy reduction with high deadline reliability.

Finally, the Wilcoxon signed-rank tests in Table VI provide statistical evidence for these advantages. With P-values below the 0.05 threshold and significance maintained after Bonferroni and False Discovery Rate (FDR) corrections, the results provide statistical evidence for MAHDRL's aggregated performance advantage in energy optimization and stability.

C. Ablation Experiments

To evaluate the contribution of each component, ablation experiments are conducted on four distinct variants. These include MAHDRL-TL, a two-level version that omits the host selection layer, as well as MAHDRL-NTA, NHA, and NVA, where the TA, HA, and VA are respectively replaced by random selection. Each variant undergoes 30 independent runs under identical configurations to ensure statistical reliability.

Fig. 5 illustrates the relative energy increase of these variants against the complete MAHDRL across 27 composite scenarios. MAHDRL-NHA leads to the largest deterioration, peaking at a 27.40% energy penalty in Loose SL. This indicates the important role of the host agent in pruning the vast decision space. Similarly, MAHDRL-TL and MAHDRL-NVA consistently exhibit energy increases, suggesting the benefit of hierarchical decoupling and fine-grained VM allocation. Furthermore, while MAHDRL-NTA performs adequately in

TABLE V
ENERGY CONSUMPTION AND SATISFACTION PROBABILITY ACROSS VARYING RESOURCE SCALES AND DEADLINES.

DDL	Scenario	ERTF-H-FIFS		PD3QN		MARL		IRWS		MAHDRL	
		EC	ESP	EC	ESP	EC	ESP	EC	ESP	EC	ESP
Loose	SS	322257.64	100%	317920.38	100%	312339.69	100%	320236.34	90%	**310750.03**	100%
	SM	369822.06	100%	361446.92	100%	357209.25	100%	333211.63	100%	**314137.81**	100%
	SL	433136.97	100%	411540.49	100%	458530.66	100%	396666.04	100%	**322505.11**	100%
	MS	506804.15	100%	503354.29	100%	**492347.68**	100%	517066.82	43.33%	501546.19	100%
	MM	539210.24	100%	526557.08	100%	534306.22	100%	505730.67	100%	**496615.86**	100%
	ML	586336.04	100%	573988.19	100%	605988.10	100%	538874.54	100%	**520598.96**	100%
	LS	810415.14	100%	805313.13	100%	**782687.98**	100%	833147.41	86.67%	804567.93	100%
	LM	828322.49	100%	810911.68	100%	830788.47	100%	823829.82	76.67%	**808656.14**	100%
	LL	866153.32	100%	836230.86	100%	889778.28	100%	825074.99	100%	**815465.02**	100%
Medium	SS	322257.64	100%	317707.46	100%	315079.66	100%	318408.02	90%	**310158.20**	100%
	SM	369822.06	100%	360714.53	100%	358202.15	100%	330397.45	100%	**325177.49**	100%
	SL	433136.97	100%	411471.01	100%	436595.24	100%	395562.62	100%	**330382.94**	100%
	MS	506804.15	100%	503120.46	100%	**493838.40**	100%	512153.34	30%	500942.63	100%
	MM	539210.24	100%	525897.37	100%	526300.36	100%	503900.15	100%	**502952.96**	100%
	ML	586336.04	100%	574247.20	100%	593159.68	100%	536000.74	100%	**527234.71**	100%
	LS	810415.14	100%	805055.34	100%	**786180.59**	100%	825359.91	73.33%	804642.27	100%
	LM	828322.49	100%	811990.02	100%	834510.29	100%	837669.61	60%	**810019.56**	100%
	LL	866153.32	100%	834786.93	100%	898590.72	100%	827277.13	100%	**816521.51**	100%
Tight	SS	322257.64	100%	317965.13	100%	312896.34	100%	315483.64	70%	**309986.84**	100%
	SM	369822.06	100%	360512.60	100%	357295.06	100%	334727.38	100%	**321647.93**	100%
	SL	433136.97	100%	411964.79	100%	443040.54	100%	400034.27	100%	**324051.56**	100%
	MS	506804.15	100%	503222.75	100%	**491838.91**	100%	507573.32	20%	500851.52	100%
	MM	539210.24	100%	525918.01	100%	531664.58	100%	509760.32	100%	**501806.34**	100%
	ML	586336.04	100%	573968.30	100%	601923.90	100%	541658.82	100%	**522417.45**	100%
	LS	810415.14	100%	805544.54	100%	**781779.31**	100%	816115.00	70%	804402.33	100%
	LM	828322.49	100%	812203.03	100%	833051.35	100%	831075.06	36.67%	**810063.11**	100%
	LL	866153.32	100%	834348.89	100%	898612.03	100%	831547.52	100%	**818459.50**	100%

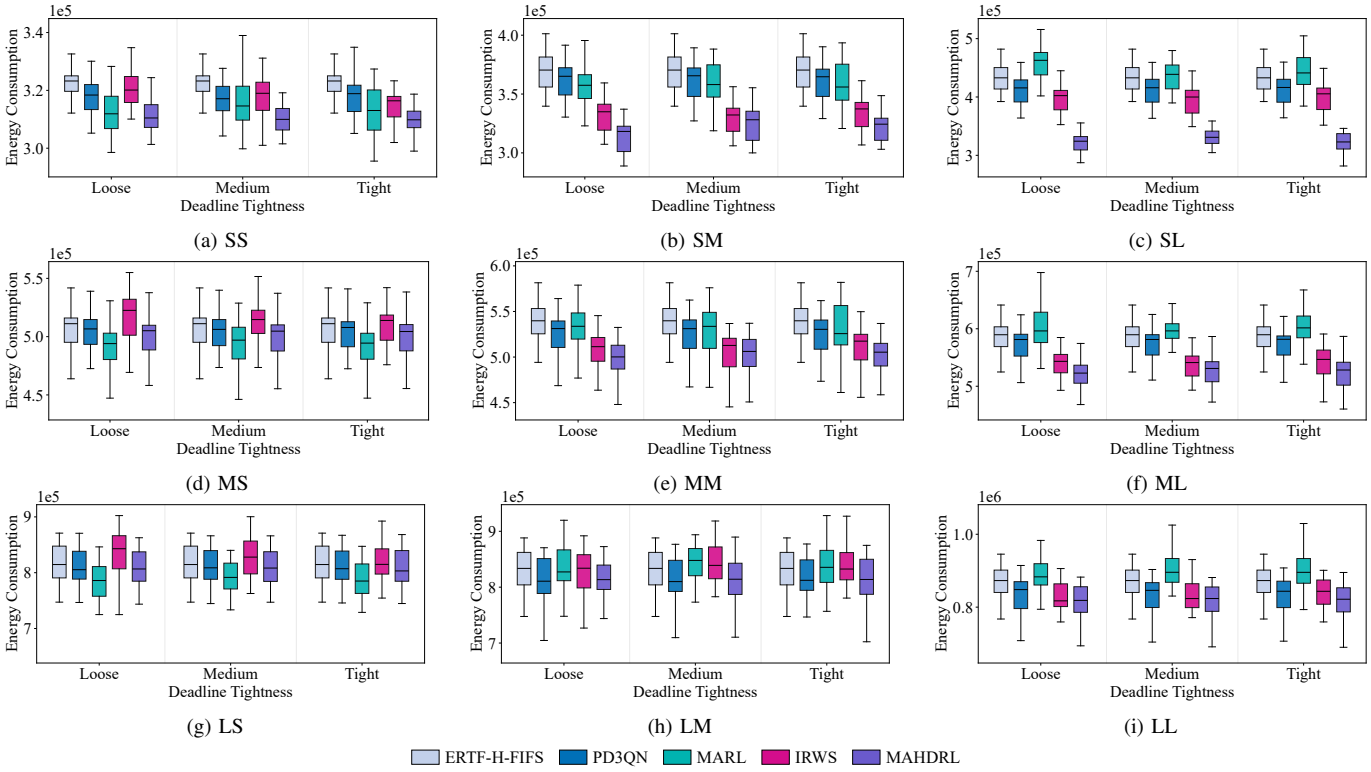


Fig. 4. Energy consumption distributions of evaluated algorithms under varying resource scales and deadlines.

TABLE VI
RESULTS OF WILCOXON SIGNED-RANK TESTS.

Algorithm	P-Value	Bonferroni	FDR
ERTF-H-FIFS	7.45E-09	2.98E-08	9.93E-09
PD3QN	7.45E-09	2.98E-08	9.93E-09
MARL	6.27E-05	2.56E-04	6.27E-05
IRWS	7.45E-09	2.98E-08	9.93E-09

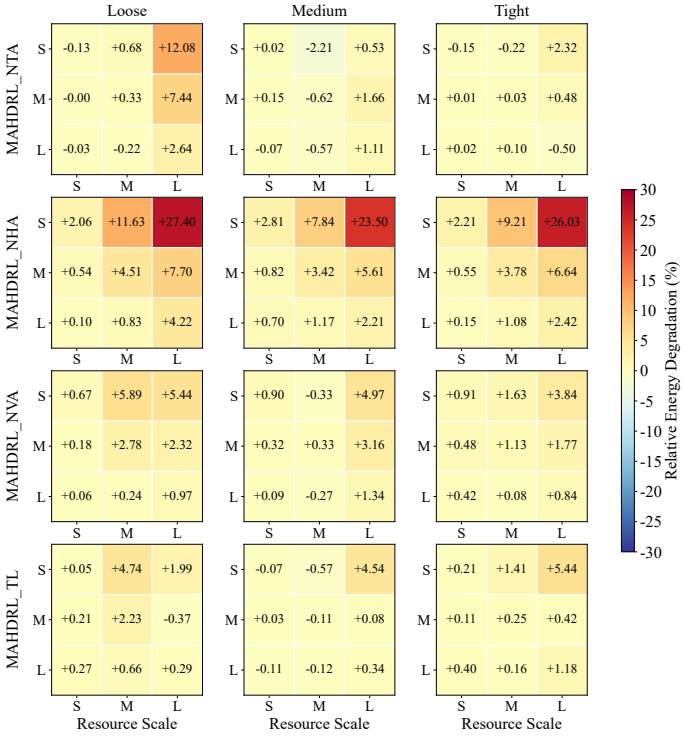


Fig. 5. Relative energy degradation of ablated configurations.

balanced settings, it degrades noticeably in unbalanced configurations, yielding a 12.08% penalty in the loose SL scenario. Although rare negative values, notably -2.21% for MAHDRL-NTA in Medium SM, occur due to the randomness of DRL or specific heuristic conditions, the overall results support the effectiveness of the complete MAHDRL. Ultimately, this fully coordinated architecture is essential for managing large decision spaces and improving energy efficiency.

VI. Conclusions

This study investigates dynamic energy-aware cloud workflow scheduling under deadline constraints and proposes a multi-agent hierarchical deep reinforcement learning (MAHDRL) framework to reduce decision-making complexity. MAHDRL decomposes the scheduling process into three coordinated decision tiers: dynamic task sequencing, energy-aware host selection, and fine-grained VM selection. This hierarchical architecture enables each agent to specialize in a specific sub-decision while remaining aligned with the global scheduling objective. By integrating the dueling double deep Q-network with a hierarchical asynchronous update mechanism, MAHDRL improves training stability and mitigates rapid policy fluctuations. Extensive experiments on scientific workflows show that MAHDRL achieves lower energy consumption in most tested scenarios while maintaining high empirical deadline and QoS satisfaction across diverse task scales, resource configurations, and deadline stringencies. These results demonstrate the effectiveness and robustness of the proposed hierarchical scheduling strategy.

Future work will extend MAHDRL to more complex and uncertain edge-cloud environments by considering stochastic execution latencies, dynamic resource availability, node faults, and communication delays. Another direction is to incorporate multi-objective optimization to jointly balance energy consumption, monetary cost, response time, and deadline satisfaction. In addition, improving scalability and transferability for larger-scale workflows on highly heterogeneous computing infrastructures remains a promising research avenue.

References

[1] J. Zhou, L. Gao, S. Rao, and Y. Li, "Scheduling constrained cloud workflow tasks via evolutionary multitasking optimization with adaptive knowledge transfer," *IEEE Transactions on Services Computing*, vol. 17, no. 6, pp. 4254–4266, 2024.
[2] Y. Li, Y. He, J. Lin, Z. Xu, and S. Zhang, "A reinforcement learning-based population hyper-heuristic for energy-efficient cloud workflow scheduling problem," *IEEE Transactions on Services Computing*, vol. 18, no. 5, pp. 2545–2558, 2025.
[3] A. Jayanetti, S. Halgamuge, and R. Buyya, "Multi-agent deep reinforcement learning framework for renewable energy-aware workflow scheduling on distributed cloud data centers," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 4, pp. 604–615, 2024.
[4] L. Ye, L. Yang, X. Zhao, and Y. Xia, "An energy-aware multi-stages hybrid scheduling approach for iot workflow applications with reliability constraint in cloud computing systems," *IEEE Internet of Things Journal*, vol. 12, no. 15, pp. 29670–29681, 2025.
[5] Y. Xie, X. Y. Wang, Z. J. Shen, *et al.*, "A two-stage estimation of distribution algorithm with heuristics for energy-aware cloud workflow scheduling," *IEEE Transactions on Services Computing*, vol. 16, no. 6, pp. 4183–4197, 2023.
[6] Z. Sun, B. Zhang, C. Gu, R. Xie, B. Qian, and H. Huang, "ET2FA: A hybrid heuristic algorithm for deadline-constrained workflow scheduling in cloud," *IEEE Transactions on Services Computing*, vol. 16, no. 3, pp. 1807–1821, 2023.
[7] X. Cai, Y. Zhang, M. Li, *et al.*, "Dynamic deadline constrained multi-objective workflow scheduling in multi-cloud environments," *Expert Systems with Applications*, vol. 258, p. 125168, 2024.
[8] N. Rizvi, D. Ramesh, L. Wang, and A. Basava, "A workflow scheduling approach with modified fuzzy adaptive genetic algorithm in IaaS clouds," *IEEE Transactions on Services Computing*, vol. 16, no. 2, pp. 872–885, 2023.
[9] Y. Wu, S. Tang, C. Yu, B. Yang, C. Sun, J. Xiao, H. Wu, and J. Feng, "Task scheduling in geo-distributed computing: A survey," *IEEE Transactions on Parallel and Distributed Systems*, vol. 36, no. 10, pp. 2073–2088, 2025.
[10] X. Tang, W. Cao, H. Tang, T. Deng, J. Mei, Y. Liu, C. Shi, M. Xia, and Z. Zeng, "Cost-efficient workflow scheduling algorithm for applications with deadline constraint on heterogeneous clouds," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 9, pp. 2079–2092, 2022.
[11] L. Zhao, J. Fan, C. Zhang, W. Shen, and J. Zhuang, "A DRL-based reactive scheduling policy for flexible job shops with random job arrivals," *IEEE Transactions on Automation Science and Engineering*, vol. 21, no. 3, pp. 2912–2923, 2024.
[12] J. Chi, X. Zhou, F. Xiao, *et al.*, "Task offloading via prioritized experience-based double dueling DQN in edge-assisted IIoT," *IEEE Transactions on Mobile Computing*, vol. 23, no. 12, pp. 14575–14591, 2024.
[13] K. Zhu, Z. Zhang, S. Zeadally, and F. Sun, "Learning to optimize workflow scheduling for an edge–cloud computing environment," *IEEE Transactions on Cloud Computing*, vol. 12, no. 3, pp. 897–912, 2024.
[14] Z. Li, Q. Ji, X. Ling, and Q. Liu, "A comprehensive review of multiagent reinforcement learning in video games," *IEEE Transactions on Games*, vol. 17, no. 4, pp. 873–892, 2025.

[15] S. Absa, A. S. Radhamani, and Y. M. Reeja, "Multi-objective optimization based efficient resource scheduling scheme for infrastructure as a service cloud computing," *Swarm and Evolutionary Computation*, vol. 99, p. 102168, 2025.

[16] G. Fan, X. Chen, Z. Li, H. Yu, and Y. Zhang, "An energy-efficient dynamic scheduling method of deadline-constrained workflows in a cloud environment," *IEEE Transactions on Network and Service Management*, vol. 20, no. 3, pp. 3089–3103, 2023.

[17] L. Zhang, M. Ai, K. Liu, J. Chen, and K. Li, "Reliability enhancement strategies for workflow scheduling under energy consumption constraints in clouds," *IEEE Transactions on Sustainable Computing*, vol. 9, no. 2, pp. 155–169, 2024.

[18] X. Lin, Y. Wang, Q. Xie, and M. Pedram, "Task scheduling with dynamic voltage and frequency scaling for energy minimization in the mobile cloud computing environment," *IEEE Transactions on Services Computing*, vol. 8, no. 2, pp. 175–186, 2014.

[19] Z. Tang, L. Qi, Z. Cheng, K. Li, S. U. Khan, and K. Li, "An energy-efficient task scheduling algorithm in DVFS-enabled cloud environment," *Journal of Grid Computing*, vol. 14, no. 1, pp. 55–74, 2016.

[20] V. Singh, I. Gupta, and P. K. Jana, "An energy efficient algorithm for workflow scheduling in IaaS cloud," *Journal of Grid Computing*, vol. 18, no. 3, pp. 357–376, 2020.

[21] X. Mei, X. Chu, H. Liu, Y.-W. Leung, and Z. Li, "Energy efficient real-time task scheduling on CPU-GPU hybrid clusters," in *IEEE Conference on Computer Communications (INFOCOM)*, pp. 1–9, 2017.

[22] Q. Wang, X. Mei, H. Liu, Y.-W. Leung, Z. Li, and X. Chu, "Energy-aware non-preemptive task scheduling with deadline constraint in DVFS-enabled heterogeneous clusters," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 12, pp. 4083–4099, 2022.

[23] A. Tzenetopoulos, D. Masouros, S. Xydis, and D. Soudris, "Ωkypous: Harnessing timing slacks and coordinated DVFS for power-efficient serverless workflows," *IEEE Transactions on Computers*, 2026. Early Access.

[24] T. Wu, H. Gu, J. Zhou, T. Wei, X. Liu, and M. Chen, "Soft error-aware energy-efficient task scheduling for workflow applications in dvfs-enabled cloud," *Journal of Systems Architecture*, vol. 84, pp. 12–27, 2018.

[25] R. Garg, M. Mittal, and L. H. Son, "Reliability and energy efficient workflow scheduling in cloud environment," *Cluster Computing*, vol. 22, no. 4, pp. 1283–1297, 2019.

[26] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 13, no. 3, pp. 260–274, 2002.

[27] H. Arabnejad and J. G. Barbosa, "List scheduling algorithm for heterogeneous systems by an optimistic cost table," *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, no. 3, pp. 682–694, 2013.

[28] L. Yang, Y. Xia, L. Ye, R. Gao, and Y. Zhan, "A fully hybrid algorithm for deadline constrained workflow scheduling in clouds," *IEEE Transactions on Cloud Computing*, vol. 11, no. 3, pp. 3197–3210, 2023.

[29] Z. Chen, J. Hu, G. Min, C. Luo, and T. El-Ghazawi, "Adaptive and efficient resource allocation in cloud datacenters using actor-critic deep reinforcement learning," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 8, pp. 1911–1923, 2022.

[30] J. Zhang, Z. Ning, M. Waqas, H. Alasmary, S. Tu, and S. Chen, "Hybrid edge-cloud collaborator resource scheduling approach based on deep reinforcement learning and multiobjective optimization," *IEEE Transactions on Computers*, vol. 73, no. 1, pp. 192–205, 2024.

[31] L. Lin, L. Pan, and S. Liu, "SpotDAG: An RL-based algorithm for DAG workflow scheduling in heterogeneous cloud environments," *IEEE Transactions on Services Computing*, vol. 17, no. 5, pp. 2904–2917, 2024.

[32] C. Liu, F. Tang, Y. Hu, K. Li, Z. Tang, and K. Li, "Distributed task migration optimization in MEC by extending multi-agent deep reinforcement learning approach," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 7, pp. 1603–1614, 2021.

[33] Y. Li *et al.*, "Task placement and resource allocation for edge machine learning: A GNN-based multi-agent reinforcement learning paradigm," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 12, pp. 3073–3089, 2023.

[34] C. Sun, X. Li, C. Wang, Q. He, X. Wang, and V. C. M. Leung, "Hierarchical deep reinforcement learning for joint service caching and computation offloading in mobile edge-cloud computing," *IEEE Transactions on Services Computing*, vol. 17, no. 4, pp. 1548–1564, 2024.

[35] C. Ling, K. Peng, S. Wang, X. Xu, and V. C. M. Leung, "A multi-agent DRL-based computation offloading and resource allocation method with attention mechanism in MEC-enabled IIoT," *IEEE Transactions on Services Computing*, vol. 17, no. 6, pp. 3037–3051, 2024.

[36] Y. He, Z. Xu, J. Lin, *et al.*, "Deep reinforcement learning with evolved actions for dynamic workflow scheduling in distributed fog computing," *Neurocomputing*, vol. 677, p. 133115, 2026.

[37] S. Bharathi, A. Chervenak, E. Deelman, G. Mehta, M.-H. Su, and K. Vahi, "Characterization of scientific workflows," in *2008 Third Workshop on Workflows in Support of Large-Scale Science*, pp. 1–10, IEEE, 2008.

[38] G. Juve, A. Chervenak, E. Deelman, S. Bharathi, G. Mehta, and K. Vahi, "Characterizing and profiling scientific workflows," *Future Generation Computer Systems*, vol. 29, no. 3, pp. 682–692, 2013.

[39] J. Liu, J. Ren, W. Dai, D. Zhang, P. Zhou, Y. Zhang, G. Min, and N. Najjari, "Online multi-workflow scheduling under uncertain task execution time in IaaS clouds," *IEEE Transactions on Cloud Computing*, vol. 9, no. 3, pp. 1180–1194, 2019.

[40] K. Cai, Q. Wu, M. Zhou, C. Chen, J. Wen, and S. Wang, "Dynamically scheduling deadline-constrained interleaved workflows on heterogeneous computing systems," *IEEE Transactions on Services Computing*, vol. 18, no. 2, pp. 758–769, 2025.